

AgriChain v2 — Complete beginner guide

From Farm to Fork — Verified, Transparent and Traceable.

This PDF tells you exactly what to install, which folder to open, which commands to type, which URL to click, and how the frontend talks to the backend. Follow the steps in order. Do not skip.

1. What you are building

AgriChain has two programs that must run at the same time: (1) Backend API on port 8000 — Python FastAPI, database, blockchain, AI, IoT. (2) Frontend website on port 5173 — React dark dashboard. The website calls the API. If only one is running, login will fail.

2. Install software (one time)

macOS / Linux

Install Python 3.11+ from python.org. Install Node.js 20+ from nodejs.org (LTS). In Terminal: `python3 --version` and `node --version` must print numbers.

Windows

Install Python from python.org and tick Add python.exe to PATH. Install Node.js LTS. Open PowerShell. If python opens the Microsoft Store, use `py -3` instead of `python` in every command below.

3. Open the project in VS Code

File → Open Folder → select the AgriChain folder (the folder that contains frontend, backend, README.md). Open Terminal → New Terminal. You should be inside AgriChain.

4. Backend setup

macOS / Linux:

```
cd backend
python3 -m venv .env
source .env/bin/activate
pip install -r requirements.txt
```

Windows PowerShell:

```
cd backend
py -3 -m venv .env
.\.env\Scripts\Activate.ps1
pip install -r requirements.txt
```

If Activate.ps1 is blocked: `Set-ExecutionPolicy -Scope Process RemoteSigned`

Copy environment file from project root:

```
cd ..
copy .env.example .env (Windows)
cp .env.example .env (macOS/Linux)
```

You can leave the default SECRET_KEY for local demo. Never use it on the public internet.

5. Start the backend (Terminal 1)

```
cd backend
source .venv/bin/activate (Windows: .venv\Scripts\activate)
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Wait until you see Application startup complete. First start creates the database and mines demo blocks (30–90 seconds). Then open <http://127.0.0.1:8000/docs> — this is Swagger. If it opens, backend works. <http://127.0.0.1:8000/api/health> should show status ok and chain_valid true.

6. Frontend setup (Terminal 2 — NEW terminal)

```
cd frontend
npm install
npm run dev
```

Open <http://127.0.0.1:5173> — dark landing page. Vite proxies /api to port 8000, so you do not type the API URL yourself.

7. Login (how frontend connects to backend)

Click Login. Email admin@agrichain.local password Demo@12345. The site POSTs JSON to /api/auth/login. Backend checks the password hash, returns a JWT. The site stores the token and sends Authorization: Bearer ... on every later request. If login fails: backend not running, wrong password, or CORS. Farmer account cannot open Users page (403) — that is correct RBAC.

8. Click-by-click demo for judges

- 1 Landing page: read hero, animated farm-to-fork steps, hash ticker.
- 2 Login as admin@agrichain.local / Demo@12345.
- 3 Dashboard: totals, temperature chart, live blockchain card.
- 4 Batches → open [RICE-KONASEEMA-2026-0001](#) → QR + timeline + AgriTrust score.
- 5 Quality page: moisture 12.4, submit, see Grade A.
- 6 Tracking: load route map (OpenStreetMap, no paid key).
- 7 IoT: Simulate reading, then Inject 89°C. Graph spikes. AI should flag anomaly.
- 8 AI Risk Center: click [MAIZE-ELURU-2026-0004](#) or the spiked rice batch. Read reasons and bar importance — this is Isolation Forest + rules, not random text.
- 9 Blockchain explorer: Verify entire chain → valid. Simulate tampering → INTEGRITY COMPROMISED and failed block number. Restore demo state.
- 10 Open [/verify/RICE-KONASEEMA-2026-0001](#) (or scan QR). Consumer page: Authentic Product, four checks, trust score. No blockchain jargon.
- 11 Documents: upload a txt, then verify same file (VERIFIED) vs edited file (COMPROMISED).
- 12 Recall: start recall on [CHILLI-GUNTUR-2026-0003](#), see downstream locations.
- 13 Hackathon Demo page: Next walks 10 presentation steps.
- 14 Logout. Login as farmer@agrichain.local. Sidebar hides admin Users/Audit.
- 15 Optional: Telugu toggle in the header.

9. Demo accounts (all password Demo@12345)

admin, farmer, collection, inspector, transporter, warehouse, processor, distributor, retailer, regulator, consumer — each is email like farmer@agrchain.local

10. How layers connect

Browser (React) → HTTP JSON → FastAPI → SQLite off-chain data + Python SHA-256 PoW chain (on-chain hashes) → IsolationForest risk → IoT simulator writes sensor_readings → QR encodes http://127.0.0.1:5173/verify/BATCH_ID. Solidity contract in /contracts is optional; BLOCKCHAIN_MODE=python is default and does not need MetaMask.

11. Tests

```
cd backend
source .venv/bin/activate
python -m pytest -v
```

12. Docker (optional)

```
From project root: docker compose up --build. UI http://127.0.0.1:8080 API
http://127.0.0.1:8000/docs
```

13. If something breaks

Port in use: close the other app or change port. ModuleNotFoundError: activate venv and pip install again. Blank UI data: backend must be on 8000. Tamper stays red: click Restore demo state. Camera QR scanner needs HTTPS or localhost permission.

14. What NOT to claim

This is a hackathon prototype. Python PoW is not Ethereum mainnet. Sustainability numbers are estimates. Price charts are recorded values, not a fairness guarantee.

API map (frontend path → backend)

POST /api/auth/login · GET /api/auth/me · GET /api/dashboard · GET/POST /api/batches · GET /api/batches/{id} · GET /api/batches/{id}/qr · POST /api/events · POST /api/quality · POST /api/sensor-data · POST /api/sensor-data/anchor · GET /api/risk/{id} · GET /api/blockchain · GET /api/blockchain/verify · POST /api/debug/tamper · POST /api/debug/restore · POST /api/documents/hash · POST /api/documents/verify · GET /api/verify/{id} · POST /api/recalls · GET /api/analytics · GET /api/health · GET /docs

You now have the full loop: Agricultural data → IoT → AI risk → blockchain integrity → QR → consumer trust.